

How we faked it

Ghani's prompt/response containers and their combinators,
in TypeScript, with CLI tools and `async` functions

September 1, 2026

What has to be faked

A container is a pair (P, R) with $P : \text{Type}$ and $R : P \rightarrow \text{Type}$, and an inhabitant of the interface is a *dependent* function $(p : P) \rightarrow Rp$ — the type of the result depends on the *value* of the argument. The combinators of §5 then build new containers out of old ones, and the sequential one, $\triangleleft$, has for its shape a Σ -type:

$$(s : \text{Shape } A) ** (\text{Pos}_A s \rightarrow \text{Shape } B)$$

TypeScript has none of this. No dependent function types, no Σ -types, no Π -types. It also has no way to make a network hop type-safe, because the wire carries JSON and JSON has no notion of a type.

So four things are faked, and this note is an account of each and of exactly where the fake leaks. The short version: the type-level fakery is sound over finite indices and costs ten casts, all in one library file; the *runtime* fakery is not sound at all, and that is unavoidable.

1 Fake 1 — the container as its graph

R cannot be a function from values to types. But a container can be presented not as the pair $(\text{Shape}, \text{Pos})$ but as its **graph**: the union of its fibres. Over a *finite* index this is an equality, not an approximation:

$$\Sigma (s : S). B s = \bigcup_{s \in S} \{s\} \times B s$$

and every container here has a finite shape type, because a shape type is a tagged union. TypeScript's conditional types distribute over unions, so the dependent lookup becomes a conditional type:

```
interface Fib<S, P> { readonly shape: S; readonly pos: P }
type Cont = Fib<unknown, unknown>;

type Shape<C extends Cont> = C extends Fib<infer S, unknown> ? S : never;

type PosAt<C extends Cont, S> =
  C extends Fib<infer S1, infer P> ? (S extends S1 ? P : never) : never;
```

A container is then written as a union of Fibs, one per prompt:

```
type AgricultureC =
  | Fib<{tag:"Harvest"; workforce: Workforce; year: number}, HarvestReport>
  | Fib<{tag:"Winter"; ruralRations: Grain; year: number}, WinterReport>
  | Fib<{tag:"Census"}, CensusReturn>;
```

Where this is honest. `Fib` is *never constructed*. There is no value of that type anywhere in the program; it is a type-level record — the object language of $\text{Fam}(\text{Set}^{\text{op}})$ — and exists only to be taken apart by `infer`. Nothing is being simulated at runtime. The union genuinely *is* the graph of R .

Two traps, both silent

The direction of the fibre test. The obvious spelling of `PosAt` is `Extract<C, Fib<S, unknown>`, which keeps the fibres assignable *to* the query. But a shape you actually hold is usually *narrower* than its fibre’s declared shape — a literal tag, a specific type. What is wanted is the fibre this shape lies in: `S extends S1`, not the reverse. Written the wrong way round, every position silently becomes `never`, and nothing complains anywhere near the mistake.

Conditional types are not inference sites. If `C` occurs only inside `Shape<C>` and `PosAt<C, S>`, TypeScript cannot infer it. `seqC(a, b)` then quietly infers `A = Cont`, every downstream position becomes `unknown`, and again there is no error where the error is. The remedy is a phantom field that never holds anything:

```
interface Runner<C extends Cont> {
  readonly fibres?: C; // never assigned; exists so C is inferable
  readonly label: string;
  run<S extends Shape<C>>(shape: S, depth: number): Promise<PosAt<C, S>>;
}
```

2 Fake 2 — the dependent function as a generic `async` method

$(p : P) \rightarrow R p$ becomes the `run` signature above. The dependency is real but it is carried at the level of *types*, not values: `S` is inferred from the argument, and the return type `PosAt<C, S>` is computed from `S`. Prompt with a `Harvest` and you are handed a `HarvestReport`; prompt with a `Census` and you are handed a `CensusReturn`; the two are not interchangeable, and no call site casts.

What `async` adds, and what it does not. `Promise<...>` is bookkeeping for the fact that the answer arrives over a socket. It is not part of the container. A container says nothing about waiting — Ghani’s point in §3 is that what makes a program event-driven is the prompt-indexed response type, not the event loop. Strip the `await` away and the container is what remains; the `Promise` is the fake, not the structure.

3 Fake 3 — the delegate leg as a CLI tool

A morphism $(P, R) \rightarrow (Q, T)$ is a pair: $u : P \rightarrow Q$ sending control *down*, and f bringing data *up*. Here u is not a function call. It is a process:

```
delegate.ts <port> <json-prompt> <depth> # stdout is the reply, as JSON
```

That file knows nothing about any container. It takes a port, a JSON prompt and a depth, and carries the text across. The ignorance is the point: u is a map of *shapes*, and shapes are exactly what survives a wire.

This one is arguably not a fake at all. A function call pretending to be a boundary is the usual way to model this; here the boundary is real. Control genuinely leaves the process. The cost is about a fifth of a second per hop, and since Gosplan fans out to four commissariats, one ten-turn game is roughly forty spawns.

And the fake bites where the compiler cannot see. `wire.ts` locates that tool with

```
new URL("./delegate.ts", import.meta.url) // resolved at run time, never checked
```

Every file once typechecked cleanly and the first playthrough died at once — `delegate` to `:8804` exited `1` — because that file had not been copied when the library was vendored. **Types hold within a process. The moment a boundary is a filesystem path or a socket, only running it tells you the truth.**

4 Fake 4 — the combinators as values

Each combinator is a function returning a **Runner** for the composite, together with an *introduction rule* for its shapes. The introduction rules are where the typing is won: each is generic in the fibre, so the checker knows which fibre you are in.

```
tensorC(a, b).both(leftShape, rightShape) // (x) ask both, both must answer
seqC(a, b).step(first, next)              // <| the dependent pair
productC(a, b, choose).offer(aShape, bShape) // x ask both, one answers
sumC(a, b).left(shape) | .right(shape)     // + route
```

The tensor’s body is three lines, and the middle one is the whole definition rather than an optimisation:

```
const [left, right] = await Promise.all([a.run(s.left, depth), b.run(s.right, depth)]);
return { left, right } as PosAt<TensorC<A, B>, S>;
```

The sequential composition is the interesting one, because its shape carries a *function*:

```
step<SA extends Shape<A>, SB extends Shape<B>>>(
  first: SA,
  next: (r: PosAt<A, SA>) => SB,          // SA is inferred from ‘first’, so ‘next’
): {first: SA; next: (r: PosAt<A, SA>) => SB} // is typed at THAT fibre alone
```

```
const first = await a.run(s.first, depth);
const second = await b.run(s.next(first), depth); // <-- this application IS <|
```

That is the Σ -type, faked: not a pair of prompts, but a prompt together with a function from its replies to the next prompt, and **next** contextually typed at the fibre you actually gave.

The known cost. **next** must be *pure* — it is a function of the position and returns only a shape. When the game needs a decision made inside **next** to be visible afterwards, it is captured in a closure variable and read back after the run. That works, and it is flagged in the source, but it is a concession: the honest version would carry the decision in the composite shape.

5 Where the fake leaks

Ten casts, all in one file

Every **run** body must destructure a shape it knows only through a conditional type, and re-assert the result. That is two casts per combinator:

site	casts	what is asserted
leaf	2	the shape has a tag ; the decoded reply is the position
sumC	2	the shape is a tagged side; the reply is the position
tensorC	2	the shape is a pair; the pair of replies is the position
productC	2	the shape is a pair; { side , value } is the position
seqC	2	the shape is { first , next }; { first , second } is the position
total	10	all inside <code>lib/algebra.ts</code> , 260 lines

None of them is in the game. The game body has casts, but they are of a different kind entirely — as **const** on literals, and narrowing strings that a validator has already checked. **No call site in stalin ever asserts the type of a container position.** The fakery is confined, which is the most that can be claimed for it, and rather more than nothing.

The JSON boundary, which is not faked but simply unsound

JSON has six kinds of value and no notion of types. When `{"tag": "Harvest", ...}` arrives, the server has not received a **Harvest**; it has received a stranger’s assertion that this is one.

Worse, a wrong assertion usually does not crash. In JavaScript a missing property reads as `undefined` rather than raising, so a handler reading a field that was never sent returns normally, with HTTP 200, carrying a well-formed reply of exactly the type the container demands and a value that means nothing. A crash would have been the better outcome: loud, immediate, and it would name the line.

So both directions are validated rather than cast. Prompts: each server’s `parse` takes `unknown` and returns the prompt type *or null*, and 56 field checks across the five servers go through what the tag claims. Replies: each leaf carries per-shape decoders keyed by tag,

```
type Decoders<C extends Cont> = {
  [K in TagOf<Shape<C>>]: (u: unknown) => PosAt<C, Extract<Shape<C>, {tag:K}>> | null;
};
```

which is a mapped type over the tags, so **the compiler will not let you forget a fibre**. Deleting one decoder while its fibre still existed was a compile error — the fibre and its handler are one fact stated twice, and the checker insists they agree.

The leak the type system cannot even see

`seqC(a, b)` is a perfectly good *value* whether or not anybody ever applies it. The checker verifies that a composite is well-formed; it never verifies that it is *reached*. One such composite sat declared, exported in the flow label, and silently never fired through an entire release — twice, in fact, and both times it was found by writing documentation rather than by compiling.

What is real

- The dependent lookup. Over a finite index the graph presentation is an equality, and the reply type genuinely follows from the prompt.
- The game’s own rules as fibres. Which export decisions exist is a function of the harvest grade, so shipping the maximum out of a failed harvest is a *type error* rather than a validation message.
- The boundary. Control really does leave the process.
- The exhaustiveness of the decoders.

What is not

- Ten casts inside the combinators, unavoidable given conditional types.
- Everything about the wire. The type guarantees the *shape* of a reply and nothing whatever about its truth — which, in a game where every commissariat inflates its dispatches, turned out to be the subject rather than a limitation.